

SMART DAIRY

AI-Powered Dairy Management Platform

Version 1.0

Software Authors: Smart Dairy Development Team
Current Software Owner: Smart Dairy Development Team
Documentation Date: October 22, 2025

ABSTRACT

Purpose of the Software:

Smart Dairy is designed to revolutionize dairy farm management through artificial intelligence. The software helps farmers optimize cow health monitoring, personalize feed recommendations, detect diseases early, and maximize milk production efficiency through data-driven insights.

Technologies Built From:

• Backend: Python Django Framework 5.2.7 • Frontend: HTML5, CSS3, JavaScript, Bootstrap 5.3.0
• AI Integration: Google Gemini AI for computer vision and natural language processing • Database: SQLite (development), PostgreSQL (production) • Authentication: Django built-in authentication system

Platform:

Smart Dairy runs as a web-based application accessible through web browsers. It can be deployed on cloud platforms (AWS, Google Cloud, Azure) and accessed from desktop computers, tablets, and mobile devices with internet connectivity.

INTRODUCTION

Smart Dairy is an innovative web-based platform that leverages advanced artificial intelligence to transform traditional dairy farming practices. The software addresses critical challenges faced by dairy farmers including disease detection, feed optimization, and production management. Key Functionalities:

- AI-powered disease detection through image analysis of cow photos
- Personalized feed recommendations based on individual cow characteristics
- Bilingual AI assistant supporting English and Swahili languages
- Real-time health monitoring and production analytics
- User-friendly dashboard for farm management
- Secure farmer registration and authentication system

How the Software Works: Farmers register and log into the platform to access a comprehensive dashboard. They can add individual cows to their farm profile, upload photos for AI-powered health analysis, receive personalized feed recommendations, and interact with an intelligent chatbot for farming advice. The system uses machine learning algorithms to analyze data and provide actionable insights for improved dairy farm productivity.

SOFTWARE DOCUMENTATION

4.1 Main Application Structure (Django Views)

```
from django.shortcuts import render, redirect, get_object_or_404 from
django.contrib.auth import authenticate, login, logout from django.contrib.auth.models
import User from django.contrib import messages from django.contrib.auth.decorators
import login_required from django.db.models import Avg, Sum from django.http import
JsonResponse from .models import Cow, HealthRecord, FeedRecord, ProductionRecord,
DailyFeedRecord from .gemini_service import AIService import json from datetime import
datetime, timedelta def landing_page(request): return render(request,
'main/landing.html') def register_view(request): if request.method == 'POST': username
= request.POST['username'] email = request.POST['email'] password =
request.POST['password'] confirm_password = request.POST['confirm_password'] if
password != confirm_password: messages.error(request, 'Passwords do not match') return
render(request, 'main/register.html') if
User.objects.filter(username=username).exists(): messages.error(request, 'Username
already exists') return render(request, 'main/register.html') user =
User.objects.create_user(username=username, email=email, password=password)
messages.success(request, 'Registration successful! Please login.') return
redirect('login') return render(request, 'main/register.html') @login_required def
dashboard(request): user_cows = Cow.objects.filter(farmer=request.user) total_cows =
user_cows.count() context = { 'total_cows': total_cows, 'cows': user_cows[:5] } return
render(request, 'main/dashboard.html', context)
```


4.2 AI Service Integration (Gemini AI)

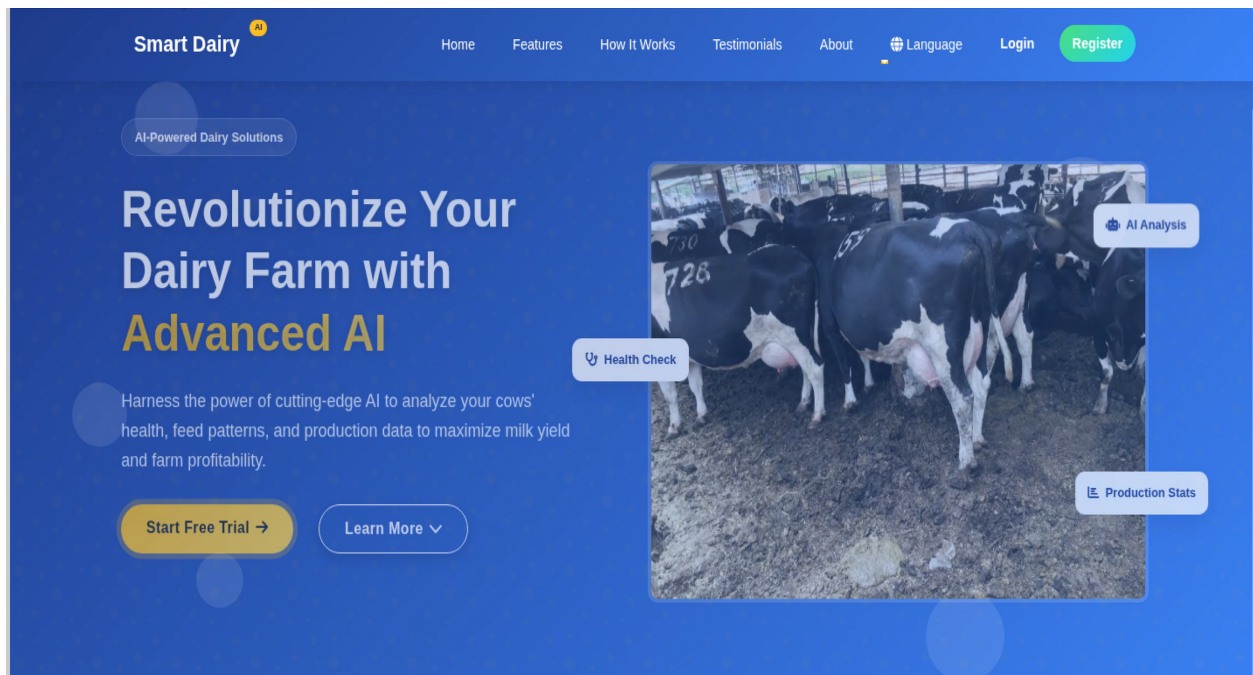
```
import google.generativeai as genai from django.conf import settings import json import re from PIL import Image as PILImage import io class AIService: def __init__(self): genai.configure(api_key=settings.GEMINI_API_KEY) self.model = genai.GenerativeModel('gemini-pro') self.vision_model = genai.GenerativeModel('gemini-pro-vision') def analyze_cow_health(self, image_file): try: image = PILImage.open(image_file) prompt = '''Analyze this cow image for health conditions. Identify any visible diseases, symptoms, and provide treatment recommendations. Format response as JSON.''' response = self.vision_model.generate_content([prompt, image]) return self.parse_health_analysis(response.text) except Exception as e: return {'error': str(e)} def generate_feed_plan(self, cow_data): prompt = f'''Create personalized feed plan for cow: Breed: {cow_data.get('breed', 'Unknown')} Weight: {cow_data.get('weight', 0)}kg Age: {cow_data.get('age', 0)} months Production: {cow_data.get('milk_yield', 0)}L/day Health: {cow_data.get('health_condition', 'Good')}''' response = self.model.generate_content(prompt) return self.parse_feed_recommendations(response.text) def chat_response(self, message, language='en'): if language == 'sw': prompt = f"Respond in Swahili about dairy farming: {message}" else: prompt = f"As a dairy farming expert, answer: {message}" response = self.model.generate_content(prompt) return response.text
```

4.3 Database Models

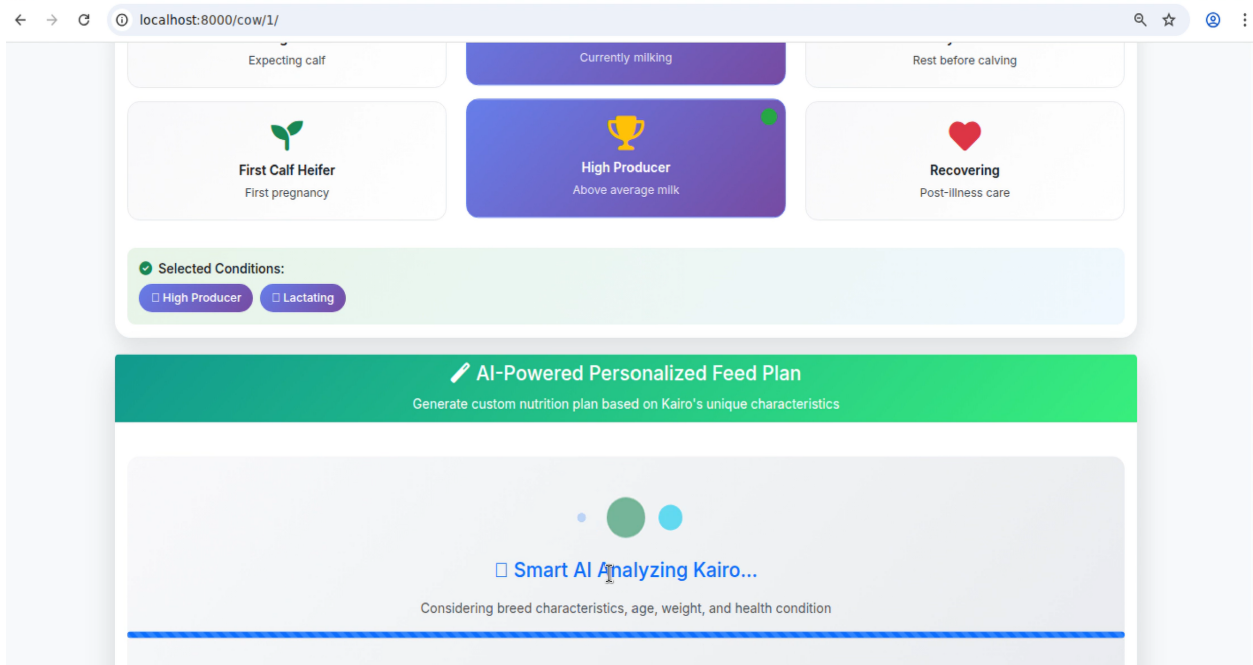
```
from django.db import models from django.contrib.auth.models import User class
Cow(models.Model): BREED_CHOICES = [ ('holstein', 'Holstein'), ('jersey', 'Jersey'),
('guernsey', 'Guernsey'), ('ayrshire', 'Ayrshire'), ('brown_swiss', 'Brown Swiss'), ]
farmer = models.ForeignKey(User, on_delete=models.CASCADE) name =
models.CharField(max_length=100) breed = models.CharField(max_length=20,
choices=BREED_CHOICES) age_months = models.IntegerField() weight = models.FloatField()
daily_milk_yield = models.FloatField(default=0) health_condition =
models.CharField(max_length=50, default='Good') date_added =
models.DateTimeField(auto_now_add=True) def __str__(self): return f"{self.name}
({self.breed})" class HealthRecord(models.Model): cow = models.ForeignKey(Cow,
on_delete=models.CASCADE) checkup_date = models.DateTimeField(auto_now_add=True)
symptoms = models.TextField() diagnosis = models.CharField(max_length=200) treatment =
models.TextField() veterinarian_notes = models.TextField(blank=True) class
ProductionRecord(models.Model): cow = models.ForeignKey(Cow, on_delete=models.CASCADE)
date = models.DateField() morning_yield = models.FloatField() evening_yield =
models.FloatField() total_yield = models.FloatField() def save(self, *args, **kwargs):
self.total_yield = self.morning_yield + self.evening_yield super().save(*args,
**kwargs)
```

4.4 User Interface Screenshots

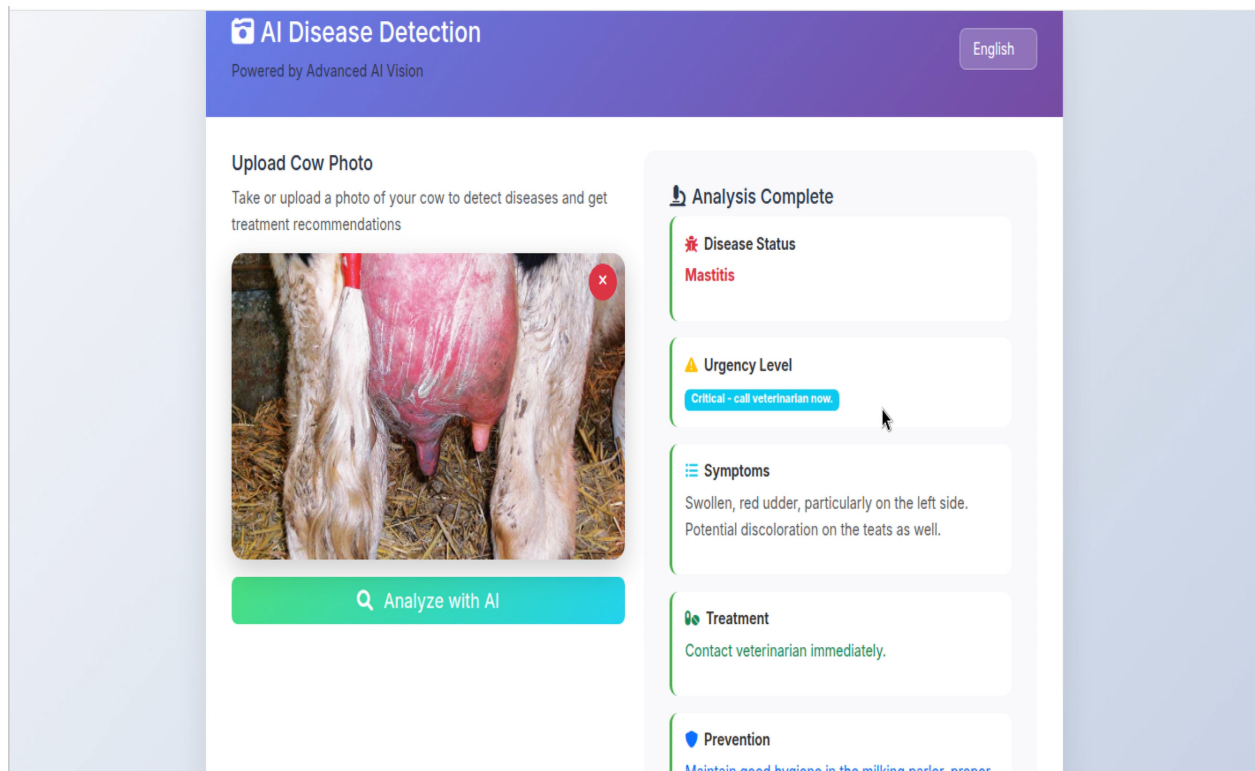
Landing Page Interface:



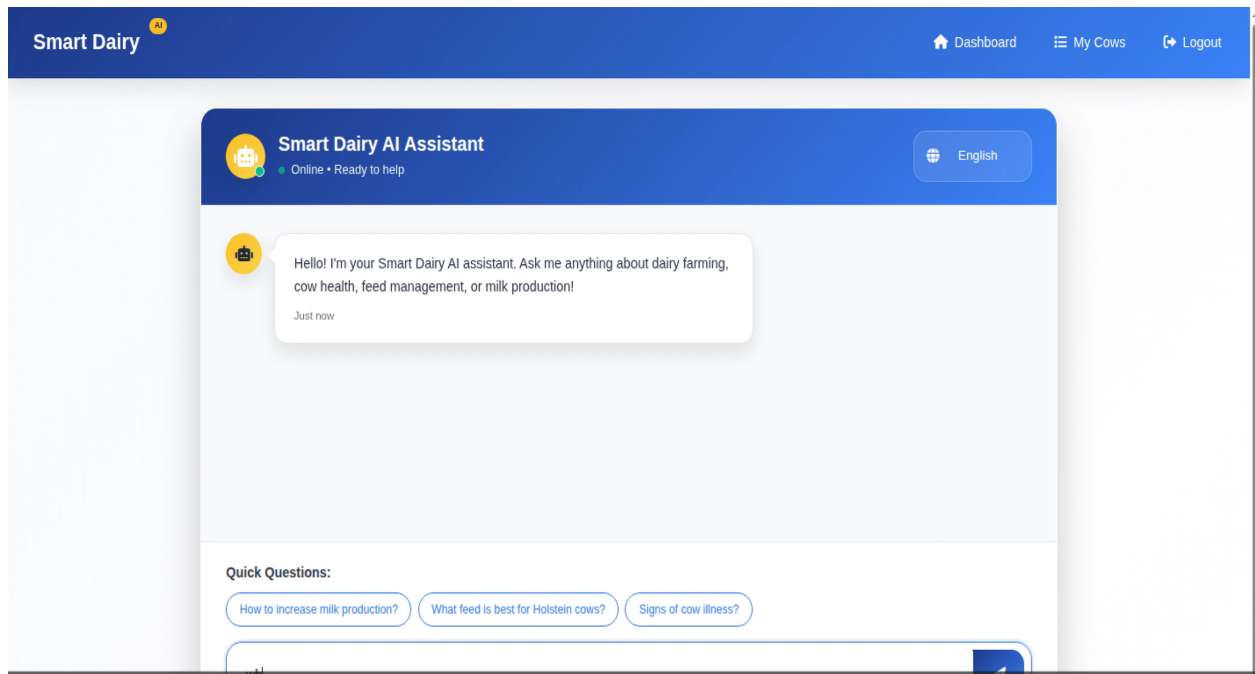
AI Feed Personalization System:



AI Disease Detection Interface:



Bilingual AI Assistant:



4.5 URL Configuration

```
from django.urls import path from . import views urlpatterns = [ path('',
views.landing_page, name='landing'), path('register/', views.register_view,
name='register'), path('login/', views.login_view, name='login'), path('logout/',
views.logout_view, name='logout'), path('dashboard/', views.dashboard,
name='dashboard'), path('add-cow/', views.add_cow, name='add_cow'), path('cow/',
views.cow_detail, name='cow_detail'), path('ai-analysis/', views.ai_analysis,
name='ai_analysis'), path('feed-plan/', views.generate_feed_plan, name='feed_plan'),
path('chat/', views.ai_chat, name='ai_chat'), path('disease-detection/',
views.disease_detection, name='disease_detection'), ]
```

Smart Dairy represents a comprehensive AI-powered solution for modern dairy farm management. The complete source code demonstrates the integration of Django web framework with Google Gemini AI to provide farmers with intelligent tools for cow health monitoring, feed optimization, and production management. The system's modular architecture ensures scalability and maintainability for future enhancements.